

Warp-Collective NTT and Keccak Instructions for ML-KEM on a RISC-V SIMT GPU

Anonymous Authors

Abstract—ML-KEM spends most of its time in two primitives whose parallelism maps naturally onto a SIMT warp but whose communication does not: a number-theoretic transform (NTT) pairs lanes at power-of-two distances, whereas a Keccak round reduces columns, gathers words, and combines row neighbors of a 5×5 state. Software must synthesize this communication from shuffles, and a memory-coupled accelerator moves the entire state through the cache on every call. In this paper we present warp-collective instructions that keep both states in ordinary lane registers of an open RISC-V GPU and expose the required communication as fixed datapaths, while preserving the standard two-source, one-destination operand contract. A pair-butterfly instruction returns both outputs of a Cooley–Tukey or Gentleman–Sande butterfly from one modular product and shares 16 multipliers across 32 lanes. Keccak is offered at two granularities, three per-stage instructions or one whole-round instruction, each producing one 32-bit half of the next state on RV32. Implemented in the Vortex core and evaluated with cycle-accurate XRT simulation and post-route Alveo V80 implementation, the collectives accelerate complete ML-KEM-768 by up to $1.583\times$ over the same shuffle-based mapping and $5.467\times$ over scalar software, for a 0.898–1.129% LUT increase over an NTT-equipped core at 250 MHz. Whole-round fusion triples permutation throughput yet improves complete requests by only 3.3%, and the memory-coupled engine overtakes register collectives at eight concurrent requests. Collective granularity must therefore be chosen at application scope.

Index Terms—post-quantum cryptography, ML-KEM, NTT, Keccak, SIMT, RISC-V, instruction-set extension

I. INTRODUCTION

Post-quantum key establishment is now mandatory in TLS and VPN deployments, and ML-KEM [1] is the standardized mechanism. A server terminating many connections sees ML-KEM as a throughput workload, which makes a GPU an attractive host. Each ML-KEM-768 request executes 140 Keccak-[1600] permutations [2] and 24 forward or inverse NTTs; on a scalar RISC-V core, Keccak alone accounts for about 66% of a request and the transforms for another 13%.

Both primitives contain enough independent work to occupy a 32-lane warp, but they communicate across the warp in different ways (Fig. 1). An NTT butterfly exchanges one coefficient with a partner lane at distance 16, 8, 4, 2 and applies a modular product; a Keccak round reduces five-word columns, permutes and rotates all 25 words, and combines three row neighbors. A SIMT core supports only lane-local arithmetic and generic shuffles, so a cooperative software mapping pays for every pattern with shuffle and select instructions: on our platform a 25-lane shuffle mapping still spends 6,753 cycles per permutation, and in a transform whose products already use a modular-multiply instruction, shuffle communication ac-

counts for about two thirds of the cycles. The usual alternative, a memory-coupled Keccak engine reached through a pointer, removes the arithmetic but transfers the 200-byte state through the load/store unit on each invocation and leaves absorb and squeeze on a single lane.

Register-coupled cryptographic extensions are well established for scalar RISC-V cores [3], [4], a 25-thread Keccak layout has been used in GPU software [5], and a warp-collective Keccak round has been described in a patent application [6]. What is missing is an open implementation that fits such collectives into the operand and writeback constraints of a real SIMT pipeline, and a controlled measurement of how large a collective should be. A larger instruction removes issue slots and shuffles, but it also adds pipeline storage, and its benefit is bounded by the work that remains outside the primitive. Concurrency changes the answer again, because a memory-coupled engine overlaps with other warps while a register collective occupies the whole warp.

In this paper we present warp-collective NTT and Keccak instructions for the open Vortex RISC-V GPU [7]. The key idea is to keep each primitive’s state in ordinary general-purpose registers, distributed across lanes in the layout that its communication graph prefers, and to implement the cross-lane edges of that graph as fixed wiring inside an execution unit that still reads two source vectors and writes one destination vector. We make three contributions.

- 1) We define a register interface that fits multi-input, multi-output cryptographic operations into standard SIMT operands: a pair butterfly whose pair-low lane supplies the twiddle and whose two outputs return through the two lanes’ own destinations, and 25-lane Keccak stage and round instructions with explicit low/high output halves on RV32. No instruction carries hidden state, memory side effects, or a second destination.
- 2) We build datapaths matched to each communication structure: a statically routed NTT unit that shares 16 multipliers across 32 lanes and saves 4.1% of core LUTs and 16 DSPs at a cycle cost below 0.041%, and two-stage and four-stage Keccak pipelines that add 1.129% and 0.898% LUTs to a routed 250 MHz core.
- 3) We evaluate complete ML-KEM-768 rather than isolated primitives. Register collectives speed up whole requests by up to $1.583\times$ over the same shuffle mapping and $5.467\times$ over scalar software. Fusing a round into one instruction triples permutation throughput but adds only 3.3% at request scope, and a pointer engine wins by $1.374\times$ at eight concurrent requests despite losing at

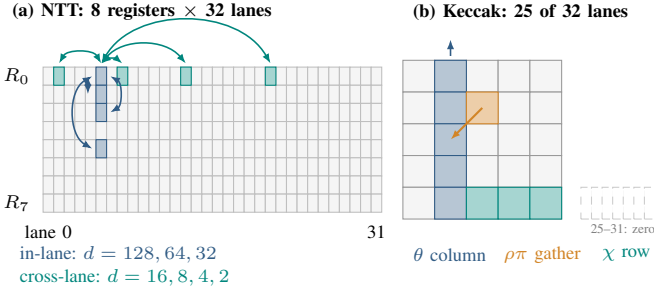


Fig. 1. Cross-lane communication of the two primitives inside one warp. (a) With coefficient $a[l + 32k]$ in register R_k of lane l , the first three NTT levels pair registers of one lane and the last four pair lanes at XOR distance d . (b) Lane $x + 5y$ holds Keccak word $A[x, y]$; θ reduces a column, $\rho\pi$ gathers and rotates a word, and χ combines row neighbors.

one. The profile that explains both results is measured directly.

II. BACKGROUND AND MOTIVATION

A. ML-KEM on a SIMT warp

ML-KEM-768 works in $\mathbb{Z}_q[X]/(X^{256} + 1)$ with $q = 3329$. Its NTT has seven levels with butterfly distances 128, 64, $\dots$, 2; each butterfly reads a pair (a, b) and a twiddle ζ and writes $(a + t, a - t)$ with $t = \text{Mont}(b\zeta)$. Hashing, sampling, and key derivation use SHAKE and SHA-3, whose Keccak-f[1600] permutation applies 24 rounds of θ , $\rho\pi$, χ , and ι to a 5×5 array of 64-bit words. One request therefore alternates between polynomial arithmetic on 256 coefficients and permutations of a 1600-bit state, with sampling, encoding, and control in between.

Vortex is an open SIMT GPU whose warps have 32 threads and whose execution units are 32 lanes wide. Every ALU-class instruction reads up to three source vectors from the register file, executes in one of several processing elements, and commits one result vector, one word per lane. The ISA offers a lane shuffle for cross-lane data movement and can be extended with custom opcodes. Our final core has eight resident warps, 16 KiB instruction and data caches, 16 KiB local memory, and no L2; the floating-point units are removed so that every reported area is an integer core.

B. Communication, not arithmetic, is the cost

Fig. 1 shows the two communication graphs. Distributing a polynomial as $R_k[l] = a[l + 32k]$ makes the first three levels lane-local: the partner of R_k is $R_{k \oplus 4}$, $R_{k \oplus 2}$, or $R_{k \oplus 1}$ in the same lane. The remaining four levels pair lane l with lane $l \oplus d$ for $d = 16, 8, 4, 2$; there is no distance-one level in ML-KEM. Keccak has no such split. With word $A[x, y]$ in lane $x + 5y$, θ needs the XOR of five lanes in a column, $\rho\pi$ moves every word to a fixed different lane and rotates it, and χ needs the two right-hand neighbors in the same row.

Software cooperative mappings can express all of these patterns with shuffles, and they already beat scalar code (Table I): spreading one state over 25 lanes completes a

TABLE I
CYCLES PER KECCAK-F[1600] PERMUTATION FOR SIX MAPPINGS ON THE SAME CORE: EIGHT WARPS, ONE STATE PER WARP, EIGHT DEPENDENT PERMUTATIONS, XRT.

Mapping	State placement	Cycles / perm.
Scalar C	one lane, memory	37,154
PQRV assembly [8]	one lane, registers	24,782
Five-lane columns	five lanes, local memory	15,910
25-lane shuffle	25 lanes, registers	6,753
Stage collectives (ours)	25 lanes, registers	390
Round collective (ours)	25 lanes, registers	120

permutation $3.7\times$ faster than optimized RV32 assembly on one lane [8], and a five-lane column mapping that exchanges words through local memory lies in between. But each shuffled Keccak round still needs dozens of instructions per lane, and a shuffled butterfly level needs a shuffle, two selects, and a product for every coefficient. Fixed wiring can implement the same edges in one instruction, provided that the instruction's inputs and outputs still fit the register file interface; the last two rows of Table I are the result.

C. Related work

Register-coupled accelerators for scalar RISC-V cores implement NTT butterflies and whole Keccak rounds inside the pipeline [3] or add modular multiply and butterfly instructions [4]; they show the value of register residence but have a single lane and no cross-lane problem. ML-Cube maps Keccak to 25 GPU threads with shuffles [5], and an NVIDIA patent application describes a warp-collective 64-bit Keccak round [6]; neither reports an implementation cost or a granularity comparison. Near-memory NTT designs choose their operation size from where butterfly partners reside [9], [10], the same principle we apply to a register file. CHAM and ZK-Flex show that primitive-level speedups in lattice and zero-knowledge accelerators shrink at application scope [11], [12], and PacQ and SynGPU show how a SIMT datapath can be extended without changing the general core [13], [14]. In contrast to all of these, we implement both a lane-pair collective and a 25-lane collective in one open GPU, keep the one-destination contract, and measure granularity, concurrency, and area together on complete ML-KEM.

III. WARP-COLLECTIVE INSTRUCTION DESIGN

A. Overview and contract

Fig. 2 places the new units in the Vortex pipeline. Three processing elements sit beside the integer ALU and multiplier: an NTT unit, a Keccak stage unit, and a Keccak round unit. Each is reached by the ordinary execute interface, which delivers the warp's source vectors and thread mask, and returns through the ordinary result interface into ALU arbitration and commit. A memory-coupled pointer engine, attached to the load/store path, is retained as the baseline for a coarser execution boundary.

All collectives obey one contract. An instruction reads two source vectors and writes one destination vector, one word per

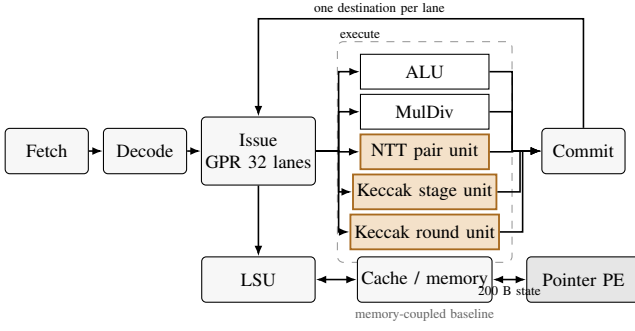


Fig. 2. The collective units are ordinary ALU processing elements of the Vortex core: they read source vectors from the register file and commit one destination vector. The pointer engine is the memory-coupled baseline.

lane. It has no pointer operand, no memory ordering effect, and no state that survives between instructions; a multi-instruction operation such as an RV32 round recomputes from identical operands rather than retaining a partial result, so warps may interleave at any instruction boundary and no context needs saving on a warp switch. Keccak collectives require all 32 lanes of the warp to be active, including the seven lanes that hold no state and return zero; NTT collectives require both lanes of every enabled pair to be active. These rules let the units use fixed wiring instead of a permutation network and keep dependence tracking, backpressure, and writeback arbitration unchanged.

B. NTT: pair butterflies with one destination per lane

A butterfly has three inputs and two outputs, so it does not fit a two-source, one-destination instruction if each lane owns one coefficient. The pair collective resolves this by letting the two lanes of a pair cooperate on one product (Fig. 3a). For a distance $d = 2^s$, lane p with bit s clear is the pair-low lane and $h = p \text{ xor } d$ is the pair-high lane. The first source vector supplies a at lane p and b at lane h ; the second source vector supplies the twiddle at lane p , and its value at lane h is ignored by definition. The Cooley–Tukey (CT) form computes

$$t = \text{Mont}(b\zeta), \quad r_p = a + t, \quad r_h = a - t, \quad (1)$$

and the Gentleman–Sande (GS) inverse form computes

$$r_p = \text{Barrett}(a + b), \quad r_h = \text{Mont}((b - a)\zeta), \quad (2)$$

with the library’s signed 16-bit wrap-around semantics. Each lane writes its own destination, so the pair’s two outputs use two ordinary writebacks and no second destination register. The instruction encodes $s \in \{0, \dots, 4\}$; ML-KEM issues eight instructions per cross-lane level and 32 per transform direction, and every twiddle is a per-lane operand rather than a table lookup inside the unit. A separate lane-local instruction, NTTMUL, performs one Montgomery product and serves the three in-lane levels, the inverse scaling, and the base multiplication of polynomial arithmetic.

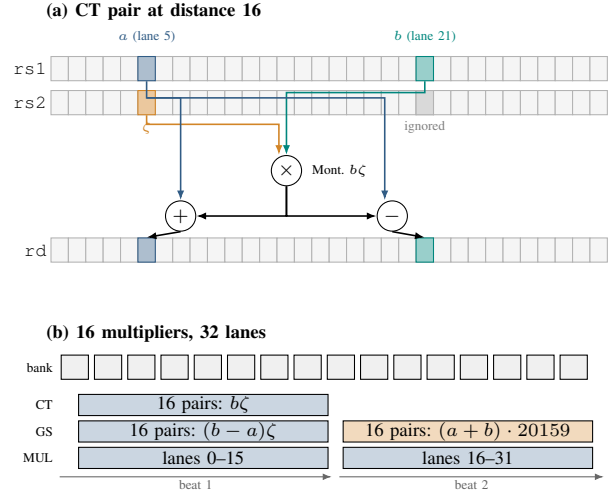


Fig. 3. (a) A CT pair butterfly at distance 16: lane 5 supplies a and the twiddle, lane 21 supplies b , one Montgomery product serves both, and each lane commits its own result. (b) Sixteen multipliers cover 32 lanes: CT needs one beat, GS needs a second beat for the Barrett quotient, and lane-local NTTMUL processes the two lane halves in turn.

C. Keccak: stage and round collectives

Lane $l = x + 5y$ holds word $A[x, y]$ as two 32-bit registers on RV32 or one register on RV64. With coordinates modulo five, one round computes

$$C[x] = \bigoplus_{y=0}^4 A[x, y], \quad (3)$$

$$T[x, y] = A[x, y] \oplus C[x-1] \oplus \text{rol}(C[x+1], 1), \quad (4)$$

$$B[y, 2x+3y] = \text{rol}(T[x, y], \rho[x, y]), \quad (5)$$

$$U[x, y] = B[x, y] \oplus (\neg B[x+1, y] \wedge B[x+2, y]), \quad (6)$$

followed by XOR of the round constant into lane zero. The *stage* design exposes three instructions, θ , $\rho\pi$, and $\chi\iota$. On RV32 each exists in a low and a high form, $\cdot_L$ and $\cdot_H$, that read the same old state vectors and return the selected half of the new word (Fig. 4a). Both halves must be computed from the full 64-bit words: the one-bit rotation in θ and the ρ rotations cross the half-word boundary, so two independent 32-bit operations would be wrong. χ operates on one half at a time and takes a warp-uniform round index as its second source. Six issues implement one round, and the programmer must not overwrite an old half before the instruction that reads it has issued, which is an ordinary register dependence.

The *round* design fuses column parity, θ , $\rho\pi$, and $\chi\iota$ into one instruction with a literal round index (Fig. 4b). On RV32, KROUND $\cdot_L$ and KROUND $\cdot_H$ each read the complete old state and compute their half of the complete new state; the pair recomputes from identical operands instead of keeping the second half in hidden storage, so different warps may interleave freely. A permutation is 48 RV32 issues instead of 144. RV64 changes only the word interface: every stage returns a whole word and one KROUND issue completes a round. Table II summarizes the collectives.

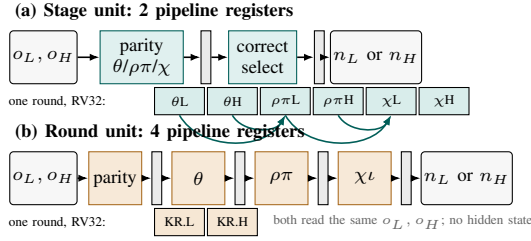


Fig. 4. Keccak collectives on RV32. (a) The stage unit computes one selected half of θ , $\rho\pi$, or χ^l through two pipeline registers; six issues form a round. (b) The round unit computes a selected half of the whole round through four pipeline registers; two issues form a round.

TABLE II

WARP-COLLECTIVE INSTRUCTIONS. ISSUES ARE PER NTT LEVEL OR PER RV32 KECCAK ROUND; RV64 HALVES THE KECCAK COUNTS.

Instruction	Lanes cooperating	Result per lane	Issues
NTTMUL	none	Montgomery product	4 / level
NTTBF.CT/GS	XOR pair, $d = 2^8$	butterfly output	8 / level
KTHETA.L/H	column of five	half of $T[x, y]$	2 / round
KRHOP.L/H	fixed source lane	half of $B[x, y]$	2 / round
KCHII.L/H	row neighbors	half of $U[x, y]$	2 / round
KROUND.L/H	all 25	half of next state	2 / round

D. Programming the complete request

The ML-KEM implementation is mlkem-native [15] compiled for Vortex. Lane zero runs the scalar control flow. Wrappers around FIPS-202, the transforms, and cooperative polynomial arithmetic activate the full warp, broadcast the leader’s arguments, execute the collective kernel, and restore the leader mask. A request-level allocation arena replaces per-lane copies of the workspace. The register-resident sponge keeps its state in lanes across absorb, permutation, and squeeze of one call; the incremental SHAKE context is saved to memory between API calls. The NTT kernel loads eight coefficients per lane, runs all seven levels in registers, and stores them back, so the transform makes no intermediate memory access.

The contract imposes three rules on kernel code. First, a Keccak collective must be reached by all 32 lanes at the same instruction; loads and stores may be guarded by `lane < 25`, but the warp must reconverge before the collective, and guarding the collective itself is a contract violation that simulation reports. Second, the round operand of χ^l must be uniform across lanes and lie in $0, \dots, 23$; the round collective encodes the index and needs no operand. Third, on RV32 the two halves of a stage must be issued before either destination overwrites an old half, which the compiler satisfies with one temporary. The permutation loop uses five general-purpose registers and contains no load, store, call, or stack frame; the expanded round version is 48 instructions with no branch.

IV. IMPLEMENTATION

A. Shared multiplier bank

A 32-lane butterfly instruction needs only 16 products, so the NTT unit instantiates 16 signed 16×16 multipliers

(Fig. 3b). Pair operands are routed to bank element i by static wiring for each of the five encoded distances, which avoids a general permutation network. CT enters the bank once. GS enters twice: the first beat forms $(b-a)\zeta$ and the second forms the Barrett quotient product $(a+b) \cdot 20159$, with the first result held in a register until both are complete. Lane-local NTTMUL also takes two beats, one per 16-lane half. With a registered reduction stage and a registered result stage, CT has a latency of six cycles and an initiation interval of one; GS and NTTMUL have seven and two. Multi-beat operations stay adjacent under backpressure and results retain their lane positions and warp metadata.

B. Keccak pipelines

The stage unit computes column parity, the $\rho\pi$ gather, or the χ row logic in its first stage and applies the θ correction or the round constant in its second; two pipeline registers give an initiation interval of one. The round unit has four registered stages, parity, θ , $\rho\pi$, and χ^l , also with initiation interval one; its stages hold complete 64-bit words, which is where its extra flip-flops come from. Padding lanes 25–31 are wired to zero. Simulation assertions check the full-mask, uniform-round, and round-range rules, and instruction-level contract tests exercise them with different twiddles per lane and partial masks.

C. Verification and physical flow

Unit tests check the butterfly against the reference for all five distances, 334,080 outputs in total, and the Keccak units against FIPS-202 for all 24 rounds, 10,200 word comparisons. Complete ML-KEM-768 requests reproduce the FIPS 203 known-answer test byte for byte for every backend. Cycle results come from XRT simulation of the accelerator function unit and processor RTL; a functional timing model (SimX) reproduces the retired instruction counts exactly and its whole-launch cycles agree within 0.17% on the matched controls. Physical results use Vivado 2025.1 for the Alveo V80 (xcv80) with a 250 MHz constraint, post-route physical optimization, and an independent full-core run per configuration.

V. EVALUATION

A. Methodology

All performance runs use one RV32IM Vortex core with eight warps of 32 threads, the cache configuration of Sec. II, and a single runtime that enables the NTT unit, both Keccak units, and the pointer engine. Only the Keccak backend of the software changes between arms; the NTT and polynomial arithmetic collectives are fixed. We compare six backends: scalar reference C, PQRV assembly [8], the 25-lane shuffle mapping (*Shuffle*), the stage collectives (*Stage*), the round collective (*Round*), and the pointer engine (*Pointer*). Each request performs key generation, encapsulation, and decapsulation with the FIPS 203 known-answer inputs. We report whole-launch device cycles for one request ($M=1$), which measures latency, and for eight concurrent requests ($M=8$), which measures batch completion time; overlapping per-request intervals are never summed.

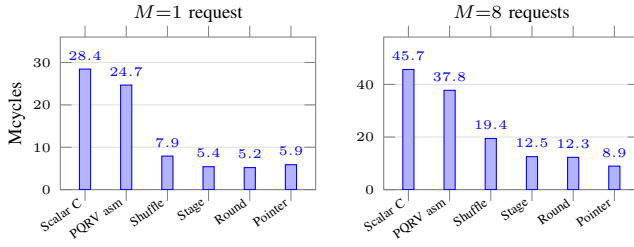


Fig. 5. Complete ML-KEM-768 whole-launch cycles on one common XRT runtime, NTT and polynomial collectives fixed, six Keccak backends. Every request passes the known-answer test. Lower is better.

B. Complete ML-KEM-768

Fig. 5 gives the headline result. The cooperative shuffle mapping alone improves on PQRV assembly by $3.120\times$ at $M=1$, so scalar software is not a fair denominator for an instruction extension. Against the identical shuffle mapping, the stage collectives improve complete requests by $1.462\times/1.552\times$ and the round collective by $1.520\times/1.583\times$ at $M=1/M=8$. Over scalar C the round backend reaches $5.467\times$ and $3.719\times$, and over PQRV $4.742\times$ and $3.074\times$; these larger ratios include the mapping change.

The best backend depends on concurrency. At one request the round collective beats the pointer engine by $1.130\times$; at eight requests the pointer engine finishes the batch $1.374\times$ faster than Round and $1.401\times$ faster than Stage. The batch gain from one to eight requests is $5.26\times$ for Pointer and $4.98\times$ for scalar C but only $3.25\text{--}3.45\times$ for the three 25-lane mappings, whose retired instructions per cycle at $M=8$ are 0.28 against 0.47 for Pointer. All three cooperative mappings issue lane-parallel sponge code that occupies the whole 32-wide datapath and load/store unit, whereas the pointer backend’s permutations execute in a separate unit and overlap with other warps. The measurements establish the crossover; attributing it to one scheduler or cache mechanism would need counters we do not have.

Instruction counts rule out the simplest explanation. At $M=8$ the round backend retires 3.33 million instructions and the pointer backend 4.22 million, so the pointer backend executes more instructions and still finishes first. Its advantage is overlap, not work: while one warp waits for the engine, the other seven issue scalar sponge and polynomial code. A register collective has no such overlap; eight 25-lane sponges contend for one 32-wide datapath, one load/store unit, and a 16 KiB data cache. The shuffle mapping, which uses no collective hardware, shows the same small batch gain, so the effect belongs to the lane-parallel software mapping rather than to the new units.

C. Granularity: primitive gains versus request gains

Round fusion cuts the RV32 issues per permutation from 144 to 48, but the round kernel is also fully unrolled because the round index is a literal. To isolate fusion we expand the stage kernel to the same 24-round straight-line code and time 64 dependent permutations per warp. Fig. 6 shows the

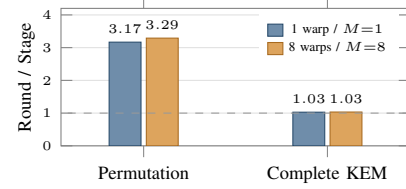


Fig. 6. Gain of the round collective over the equally expanded stage collectives. The primitive gain (64 dependent permutations per warp) does not transfer to complete ML-KEM.

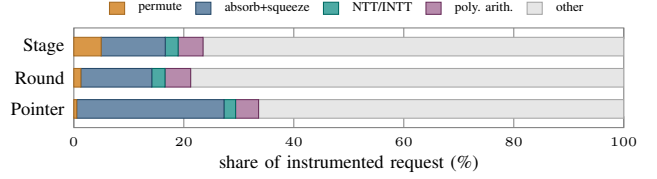


Fig. 7. Mutually exclusive intervals of one instrumented ML-KEM-768 request on XRT for the three hardware backends. Permutation is a small fraction after any of them; the pointer backend pays instead in absorb and squeeze.

outcome. At primitive scope, Round completes a permutation in 499 cycles against 1582 for expanded Stage with one active warp, and 118 against 387 per completed permutation with eight warps, a $3.168\times$ and $3.291\times$ gain. At request scope, the same two kernels in complete ML-KEM differ by $1.028\times$ at $M=1$ and $1.033\times$ at $M=8$.

Fig. 7 explains the gap with directly measured, mutually exclusive intervals of a single instrumented request. With stage collectives, permutations take 5.0% of the request; with the round collective, 1.3%; with the pointer engine, 0.5%. Absorb and squeeze, which exclude nested permutations, take 11.7% for Stage but 26.8% for Pointer, because the pointer backend runs them on one lane and moves the state through memory around every permutation. The transforms take about 2.3% and cooperative polynomial arithmetic 4.5%. The remainder, sampling, encoding, comparison, control, and wrapper overhead, is the majority of the request for every backend. Amdahl’s law with $f = 5.0\%$ and $s = 3.168$ predicts a $1/(1 - f + f/s) \approx 1.03\times$ gain from fusion, matching the measurement. Instrumentation adds 6.6–8.3% to these runs, so the headline speedups use the uninstrumented launches.

D. NTT communication and multiplier sharing

The NTT study holds the modular multiplier fixed and changes only how the four cross-lane levels communicate (Fig. 8). Replacing the scalar product with NTTMUL speeds a forward transform up by $1.37\times$; replacing shuffles with pair butterflies adds another $2.84\times$ ($2.60\times$ for the inverse), so the communication instruction is worth more than the arithmetic instruction. A stronger software control that performs the cross-lane levels through a shared-memory transpose is still $1.37\times$ slower at $M=1$ and $1.53\times$ at $M=8$. At request scope, pair butterflies reduce complete ML-KEM cycles by 2.6%/8.2% over the shuffle version and 1.0%/4.9% over the

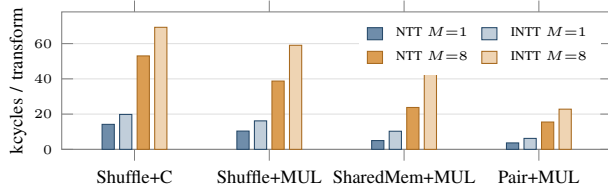


Fig. 8. Isolated 256-point forward and inverse transform cycles for one and eight concurrent transforms. *MUL* denotes the NTTMUL product, *Pair* the butterfly collective, and *SharedMem* a transpose through local memory.

TABLE III

INDEPENDENT POST-ROUTE IMPLEMENTATIONS ON THE ALVEO V80 AT 250 MHz, ONE CORE, EIGHT WARPS OF 32 THREADS, NO FLOATING POINT. KECCAK COLUMNS ARE THE UNIT HIERARCHY ALONE; WNS IS WORST NEGATIVE SLACK IN NS.

ISA	Core	LUT	FF	Keccak LUT	Keccak FF	WNS
RV32	NTT only	220,950	131,798	0	0	+0.004
RV32	+Stage	223,444	133,512	838	2,023	+0.004
RV32	+Round	222,935	138,065	2,343	6,126	+0.007
RV64	NTT only	429,067	228,266	0	0	+0.000
RV64	+Stage	432,843	232,350	1,565	3,642	-0.148
RV64	+Round	431,004	234,240	2,226	6,939	-0.013

shared-memory version at $M=1/M=8$; the transforms are a small share of a request (Fig. 7), so these are the expected magnitudes.

Sharing the multiplier bank is free at this scope. Against a matched full-core implementation with one multiplier per lane, the 16-multiplier unit removes 14,767 LUTs (4.1%), 1,565 flip-flops, and 16 DSPs, while the same binary changes complete-KEM cycles by at most 0.041%. The final NTT hierarchy occupies 5,624 LUTs, 5,155 flip-flops, and 16 DSPs.

E. Area, timing, and integer width

Table III reports independent post-route implementations of the integer-only core with the NTT unit and, in turn, each register Keccak unit. On RV32, the stage unit adds 1.129% LUTs and 1.300% flip-flops to the whole core and the round unit adds 0.898% and 4.755%. The round unit's pipeline of complete 64-bit words costs three times the flip-flops of the stage unit; its own hierarchy also has more LUTs, but the routed core is slightly smaller, so whole-core deltas include placement effects. Both close 250 MHz. In a matched sweep of an earlier core generation that still contained the floating-point units, the pointer engine added 11,221 LUTs, $5.1\times$ the increment of the round unit, so its eight-request advantage is bought with the largest area of the three backends.

A 64-bit core halves the Keccak issue count, and one might expect it to favor the collectives. Recompiling the same software for RV64IM reduces whole-launch instructions by only 7.0% (Stage) and 5.7% (Round), because NTT, sampling, and encoding do not shrink with the word size. Single-request cycles fall by 4.1% and 3.0%, eight-request cycles change by -0.4% and +0.2%, and the RV64 cores use $1.94\times$ the LUTs of their RV32 counterparts without meeting 250 MHz (Table III).

TABLE IV

DESIGN POINTS. CYCLES ARE MILLIONS PER LAUNCH ON THE COMMON RUNTIME; AREA DELTAS ARE WHOLE-CORE POST-ROUTE INCREMENTS OVER THE NTT-ONLY CORE. [†]EARLIER CORE GENERATION WITH FLOATING POINT; NOT COMPARABLE IN ABSOLUTE TERMS.

Backend	$M=1$	$M=8$	Δ LUT	Δ FF	Best for
Stage	5.41	12.53	2,494	1,714	small state, testability
Round	5.20	12.28	1,985	6,267	latency, hashing-heavy
Pointer	5.88	8.94	11,221 [†]	3,220 [†]	batched throughput

The narrow core, with its 16-bit NTT arithmetic independent of the integer width, remains the better design point.

F. Choosing a design point

Table IV collects the three hardware backends on the axes that matter to a designer. The stage collectives add the fewest flip-flops and expose three individually testable operations; they suit a register-limited core. The round collective costs 6,267 flip-flops against 1,714 but fewer LUTs, gives the lowest single-request latency, and provides about $5.1\times$ the permutation throughput per added LUT; it suits a latency-bound client or a hashing-heavy workload. The pointer engine adds several times the logic of either collective and loses at one request, but its permutations overlap with other warps and it wins the eight-request batch by $1.374\times$. None of this is visible from the permutation microbenchmark (Table I), where the round collective leads shuffles by $56.3\times$.

G. Scope and limitations

The evidence covers ML-KEM-768 at fixed known-answer inputs, two request counts, and one core geometry; rejection sampling makes other inputs shift absolute cycles for every backend alike. One state per warp is a fixed occupancy point, not an optimum; scalar and five-lane software can pack more independent states into a warp and would gain more on a hashing workload with many independent streams. All cycle results are XRT simulation of the RTL rather than board execution, and each physical row is one implementation seed. We make no energy claim, since no activity-based power measurement exists, and no side-channel claim.

VI. CONCLUSION

We presented warp-collective NTT and Keccak instructions that keep cryptographic state in the ordinary registers of a RISC-V SIMT GPU and implement each primitive's cross-lane communication as fixed wiring behind a standard two-source, one-destination interface. A pair butterfly with 16 shared multipliers and stage or round Keccak collectives add about one percent of LUTs to a routed 250 MHz core and speed up complete ML-KEM-768 by up to $1.583\times$ over the same cooperative software mapping and $5.467\times$ over scalar code. At request scope a threefold permutation gain from round fusion becomes 3.3%, and a memory-coupled engine overtakes the register collectives at eight concurrent requests. State placement, granularity, and concurrency must be chosen together at the scope of the complete request.

REFERENCES

- [1] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” NIST, Tech. Rep. FIPS 203, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [2] —, “SHA-3 standard: Permutation-based hash and extendable-output functions,” NIST, Tech. Rep. FIPS 202, 2015. [Online]. Available: <https://csrc.nist.gov/pubs/fips/202/final>
- [3] T. Fritzmann, G. Sigl, and J. Sepúlveda, “RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 4, pp. 239–280, 2020.
- [4] P. Nannipieri, S. D. Matteo, L. Zulberti, F. Albicocchi, S. Saponara, and L. Fanucci, “A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms,” *IEEE Access*, vol. 9, pp. 150 798–150 808, 2021.
- [5] T. Zhou, F. Zheng, Z. Xie, W. Tang, G. Fan, Y. Ning, Y. Bian, J. Lin, and J. Jing, “ML-Cube: Accelerating module-lattice-based cryptography using machine learning accelerators with a memory-less design,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2025, pp. 1829–1843.
- [6] M. B. Sullivan, S. Ghosh, D. R. Johnson, G. Suh, C. Joseph, and M. Y. Siu, “Warp-collective instructions to accelerate cryptographic primitives,” U.S. patent application publication US 2026/0222190 A1, Jul. 2026, applicant: NVIDIA Corp.; published July 30, 2026, Listing 3. [Online]. Available: <https://patents.justia.com/patent/20260222190>
- [7] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3d-graphics research,” in *Proc. 54th Annual IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2021, pp. 754–766.
- [8] J. Zhang, Y. Yan, J. Huang, and Ç. K. Koç, “Optimized software implementation of Keccak, Kyber, and Dilithium on RV{32,64}IM{B}{V},” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2025, no. 1, pp. 632–655, Dec. 2024.
- [9] J. Park, S. Lee, and J. Lee, “NTT-PIM: Row-centric architecture and mapping for efficient number-theoretic transform on PIM,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [10] J. Zhang, M. Imani, and E. Sadredini, “BP-NTT: Fast and compact in-SRAM number theoretic transform with bit-parallel modular multiplication,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [11] X. Ren, Z. Chen, Z. Gu, Y. Lu, R. Zhong, W. jie Lu, J. Zhang, Y. Zhang, H. Wu, X. Zheng, H. Liu, T. Chu, C. Hong, C. Wei, D. Niu, and Y. Xie, “CHAM: A customized homomorphic encryption accelerator for fast matrix-vector product,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [12] A. Putra, C. M. Duong, A. Q. Pham, and J.-Y. Kim, “ZK-Flex: A flexible and scalable framework for accelerating zero-knowledge proofs,” in *Proc. Design Automation Conf. (DAC)*, 2026.
- [13] R. Yin, Y. Li, and P. Panda, “PacQ: A SIMT microarchitecture for efficient dataflow in hyper-asymmetric GEMMs,” in *Proc. Design Automation Conf. (DAC)*, 2025.
- [14] Y. Yao, C. Zhang, C. Qi, R. Chen, J. Wang, Z. Fu, N. Jing, X. Liang, and Z. Song, “SynGPU: Synergizing CUDA and bit-serial tensor cores for vision transformer acceleration on GPU,” in *Proc. Design Automation Conf. (DAC)*, 2025.
- [15] PQ Code Package contributors, “mlkem-native,” Software repository, 2026, local dependency revision is recorded in the companion artifact. [Online]. Available: <https://github.com/pq-code-package/mlkem-native>